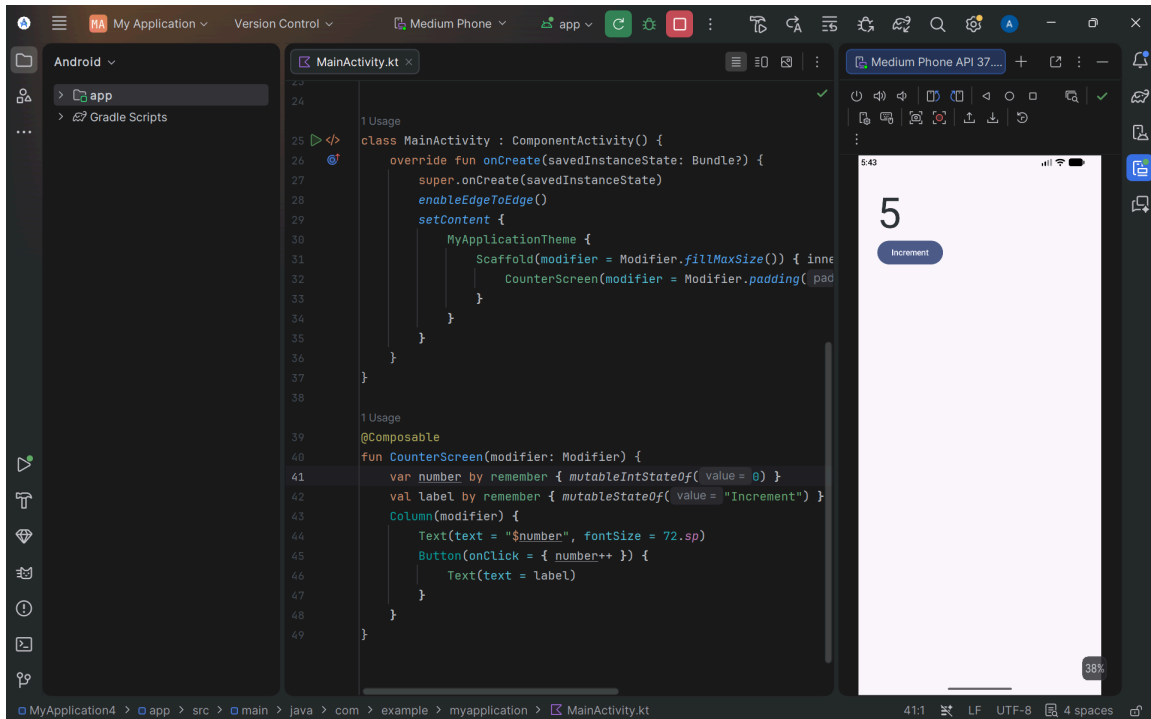


Nama : Ailsa Nasywa Hidayat
NIM : 245150407111052
Kelas : Pengembangan Aplikasi Mobile - D

MODUL 4

State Hoisting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



- a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Setiap tombol Increment diklik, angka yang ditampilkan akan bertambah 1. Contoh: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$. Jadi, jika tombol diklik lima kali dari nilai awal 0, angka yang tampil menjadi 5.

- b. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena pada Button terdapat kode:

```
Button(onClick = { number++ })
```

number++ berarti nilai number ditambah satu setiap kali tombol ditekan.

Variabel number dibuat sebagai state:

```
var number by remember { mutableStateOf(0) }
```

Karena merupakan state Compose, setiap kali nilainya berubah, Compose melakukan recomposition, yaitu memperbarui bagian UI yang menggunakan nilai tersebut.

- c. **Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?**

Variabelnya adalah:

```
number
```

Deklarasinya:

```
var number by remember { mutableStateOf(0) }
```

Kemudian nilainya ditampilkan melalui:

```
Text(  
    text = "$number",  
    fontSize = 72.sp  
)
```

Jadi Text selalu menampilkan nilai terbaru dari number.

- d. **Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?**

Variabelnya adalah:

```
label
```

Deklarasinya:

```
val label by remember { mutableStateOf("Increment") }
```

Lalu digunakan pada:

```
Text(text = "$label")
```

di dalam Button.

Pertanyaan percobaan pertama memang meminta pengamatan terhadap perubahan number dan label.

2. **Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".**

- a. **Perubahan apa yang perlu dilakukan dan dilakukan di mana?**

Pertama, ubah label tombol dari:

```
val label by remember {  
    mutableStateOf("Increment")  
}
```

menjadi:

```
val label by remember {
```

```
        mutableStateOf("Increment 2")
    }
}
```

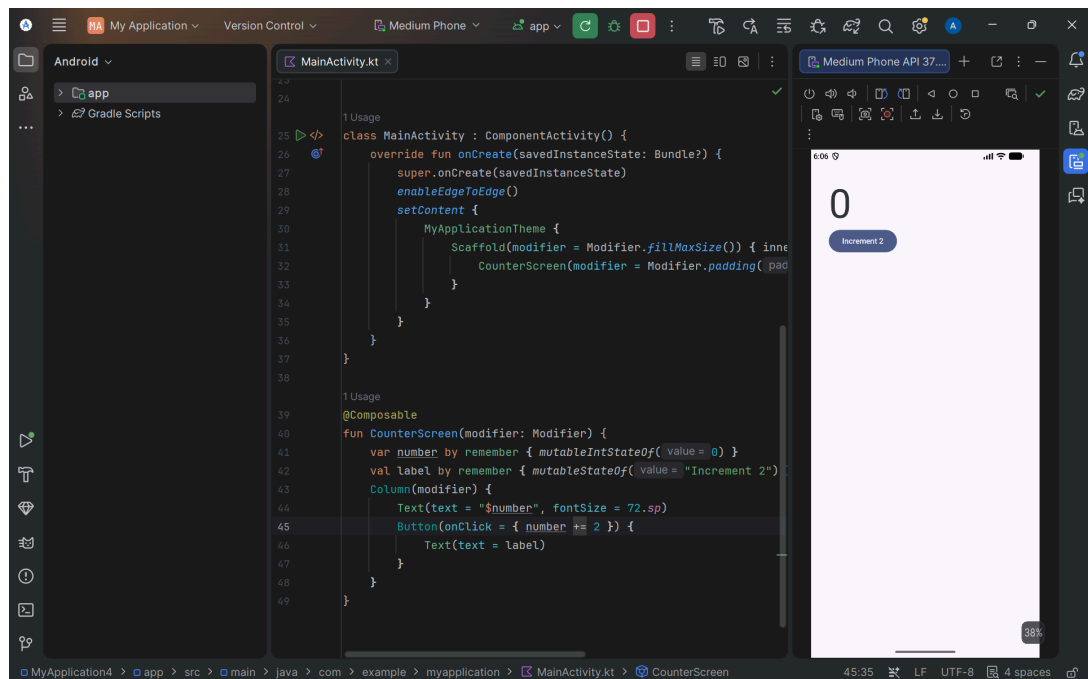
Kemudian ubah:

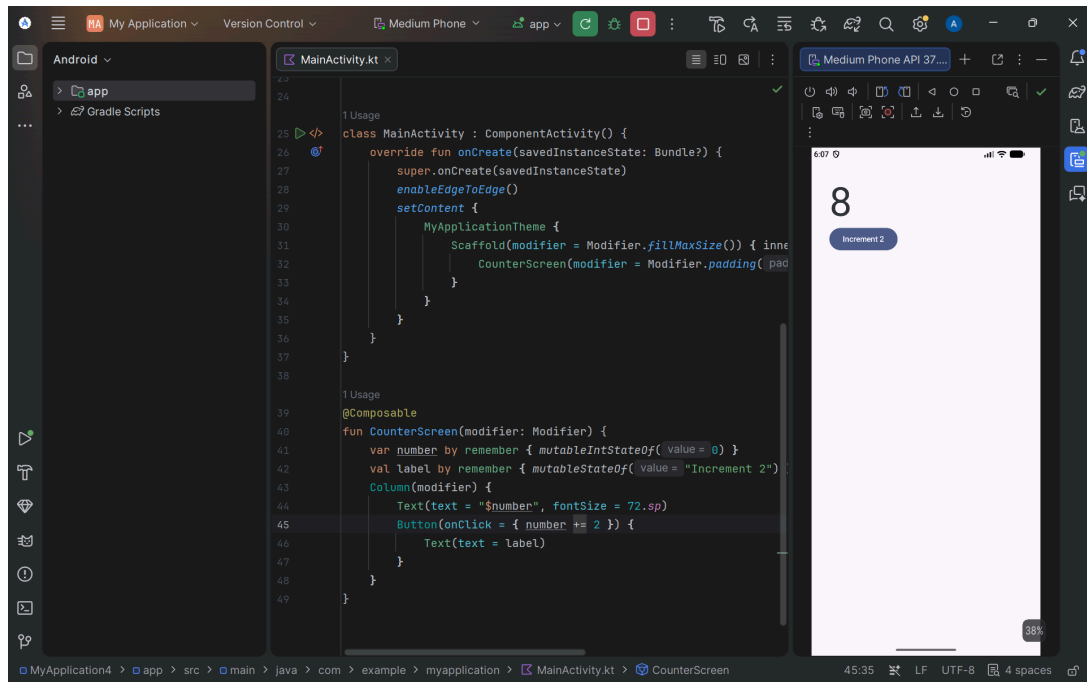
```
number++
```

menjadi:

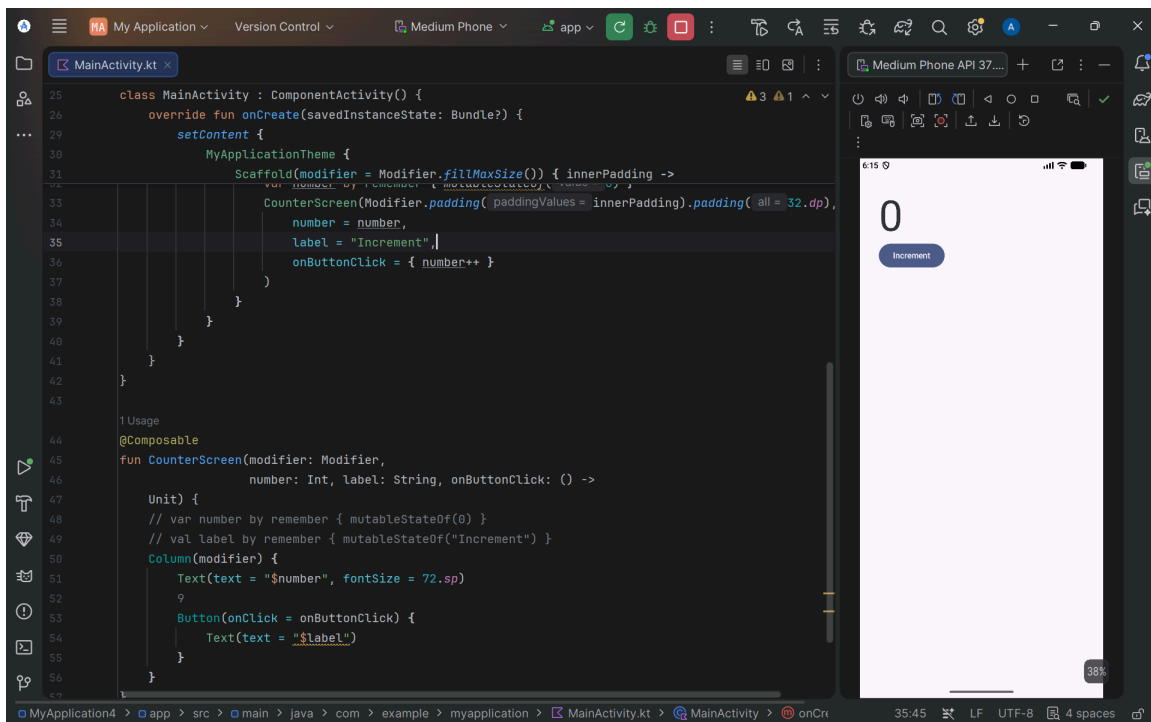
```
number += 2
```

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.





3. Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen.



4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
Ya, walaupun lokasi state sudah dipindahkan, hasil yang terlihat oleh pengguna tetap sama. Yang berbeda hanya struktur kode programnya.
 - b. Di bagian kode program manakah saat ini proses increment dilakukan?
Sekarang proses increment dilakukan di:

```
onButtonClick = {
    number++
}
```

Jadi logika perubahan data tidak lagi terdapat di dalam CounterScreen.

- c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State number berada di parent:

```
var number by remember {
    mutableStateOf(0)
}
```

Kemudian dikirim ke CounterScreen melalui:

```
number = number
```

Label tombol dikirim dengan:

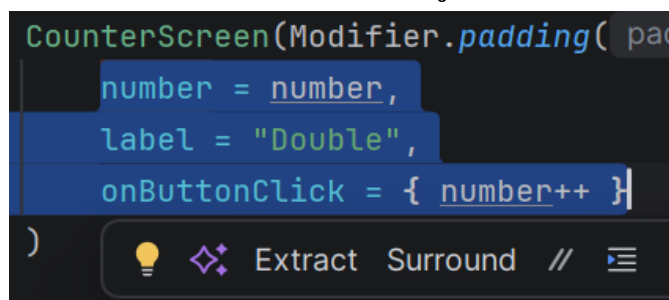
```
label = "Increment"
```

dan event klik dikirim dengan:

```
onButtonClick = {
    number++
}
```

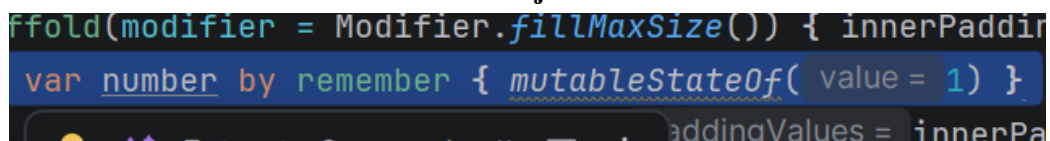
Jadi CounterScreen hanya menggunakan data yang diterimanya.

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:
- Label tombol "Increment" menjadi "Double"



```
CounterScreen(Modifier.padding(),
    number = number,
    label = "Double",
    onClick = { number++ })
```

- Nilai awal variabel number dari 0 menjadi 1.



```
var number by remember { mutableStateOf(value = 1) }
```

- Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang

dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.

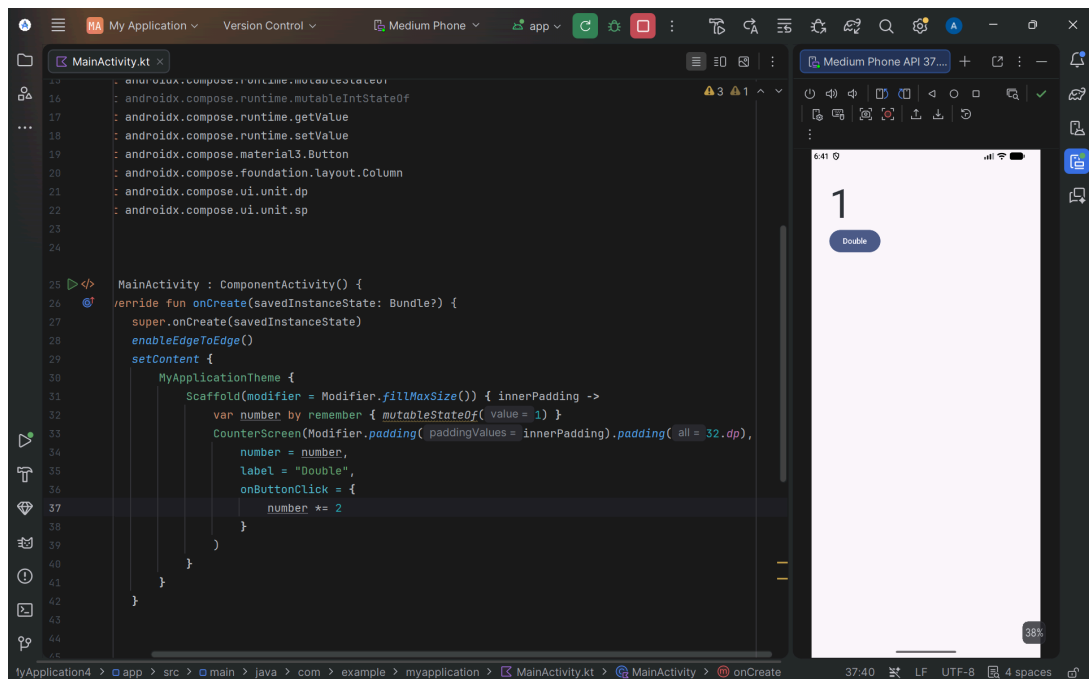
```
onButtonClick = {  
    number *= 2  
}
```

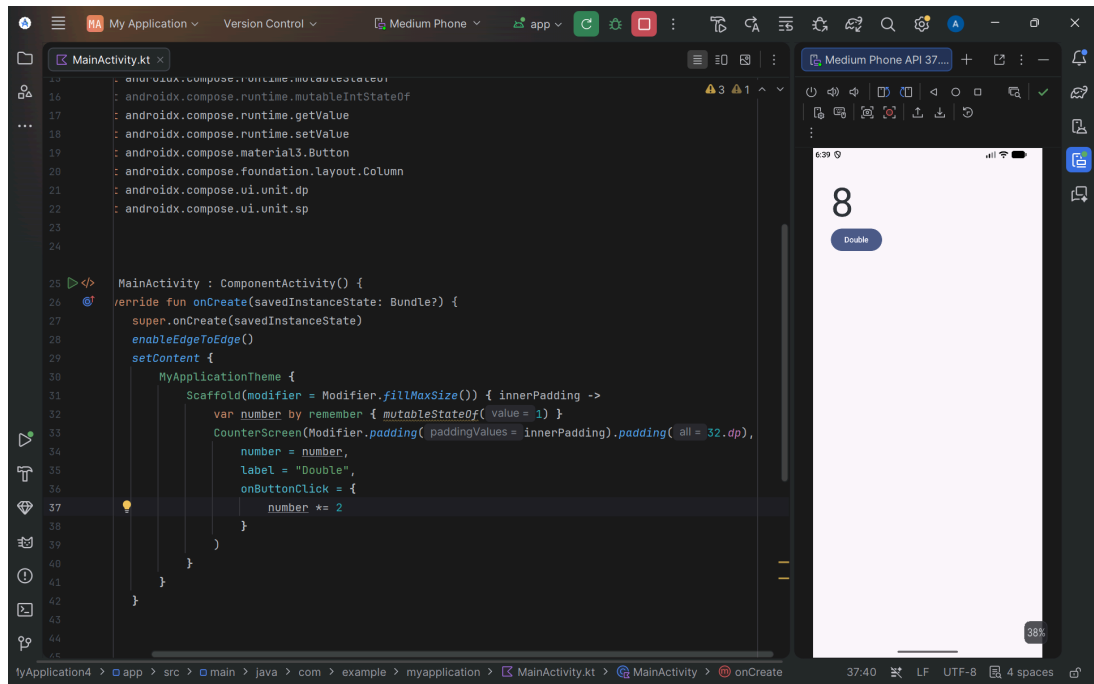
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
var number by remember { mutableStateOf( value = 1) }  
CounterScreen(Modifier.padding( paddingValues = innerPadding).padding( all = 32.dp),  
    number = number,  
    label = "Double",  
    onButtonClick = {  
        number *= 2  
    }  
)
```

6. Jalankan program

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.





- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Perubahan hanya dilakukan pada bagian parent:

```
var number by remember {
    mutableStateOf(1)
}
```

dan:

```
CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onClick = {
        number *= 2
    }
)
```

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak, fungsi Composable CounterScreen tetap. Artinya, composable yang sama bisa digunakan untuk fungsi berbeda hanya dengan mengganti data dan callback yang dikirimkan.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Dari percobaan sebelumnya terlihat bahwa perubahan fungsi:

Increment

0 → 1 → 2 → 3

menjadi:

Double

1 → 2 → 4 → 8

tidak membutuhkan perubahan pada CounterScreen. Yang diubah hanya parent. Hal ini menunjukkan manfaat utama state hoisting, yaitu:

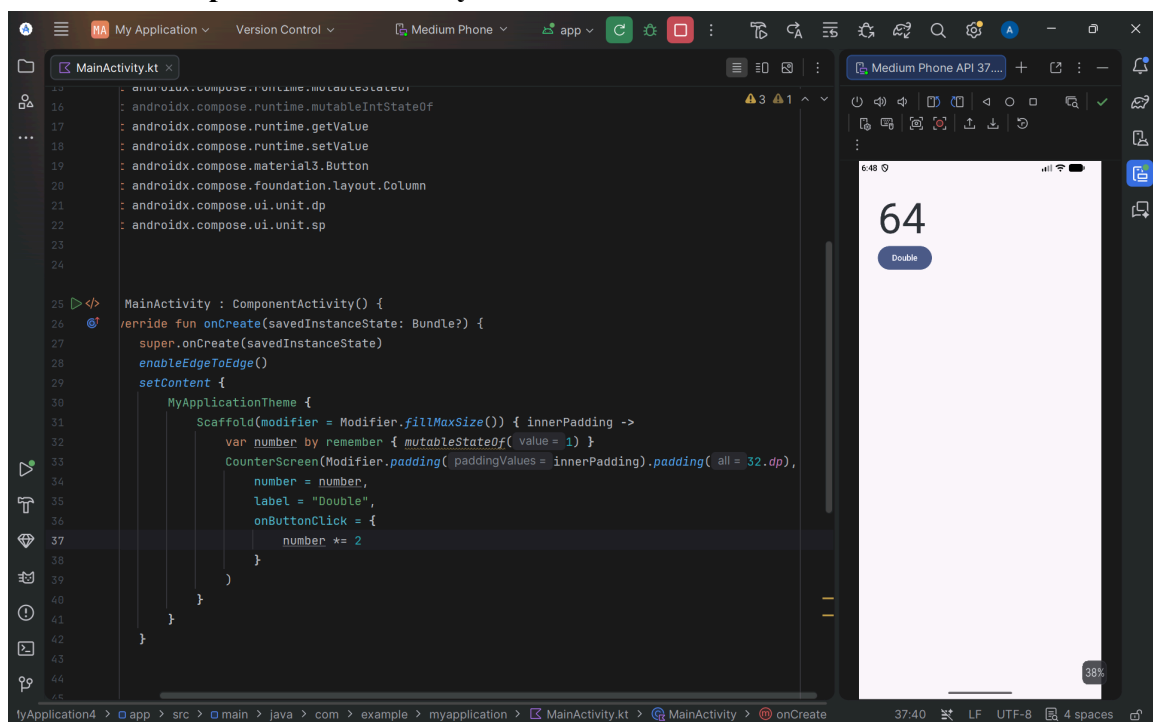
- komponen UI menjadi lebih reusable,
- logika dan tampilan lebih terpisah,
- perubahan kebutuhan lebih mudah dilakukan,
- state memiliki satu sumber utama atau Single Source of Truth,
- aliran data lebih jelas.

Pada state hoisting, state mengalir dari parent ke child, sedangkan event mengalir dari child ke parent melalui callback.

Perubahan Konfigurasi

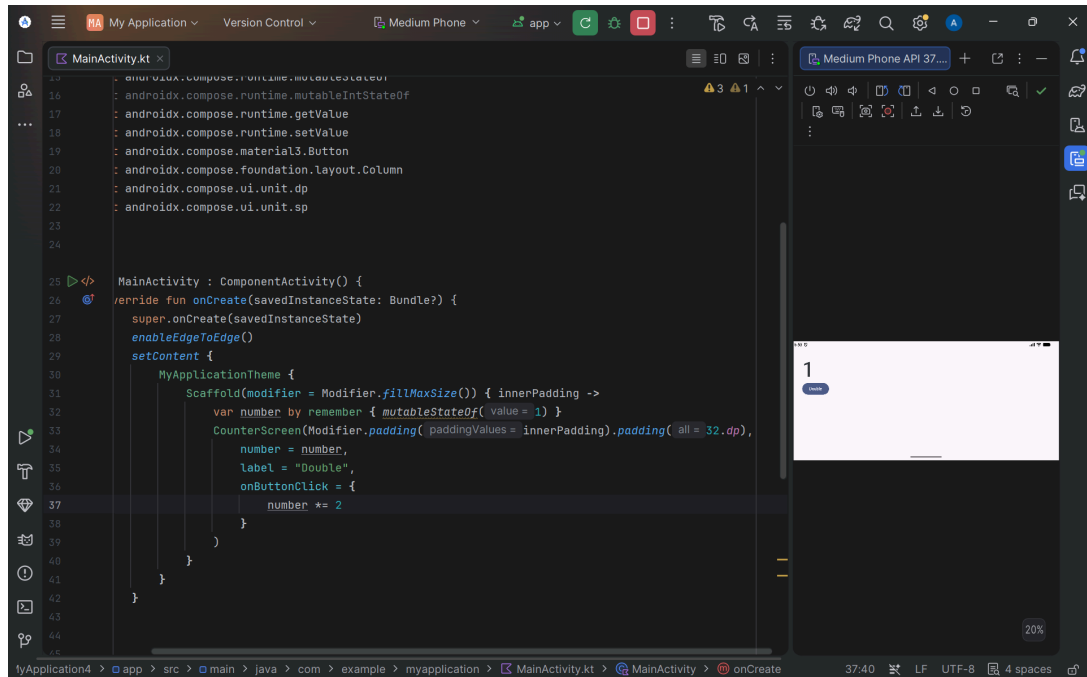
1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

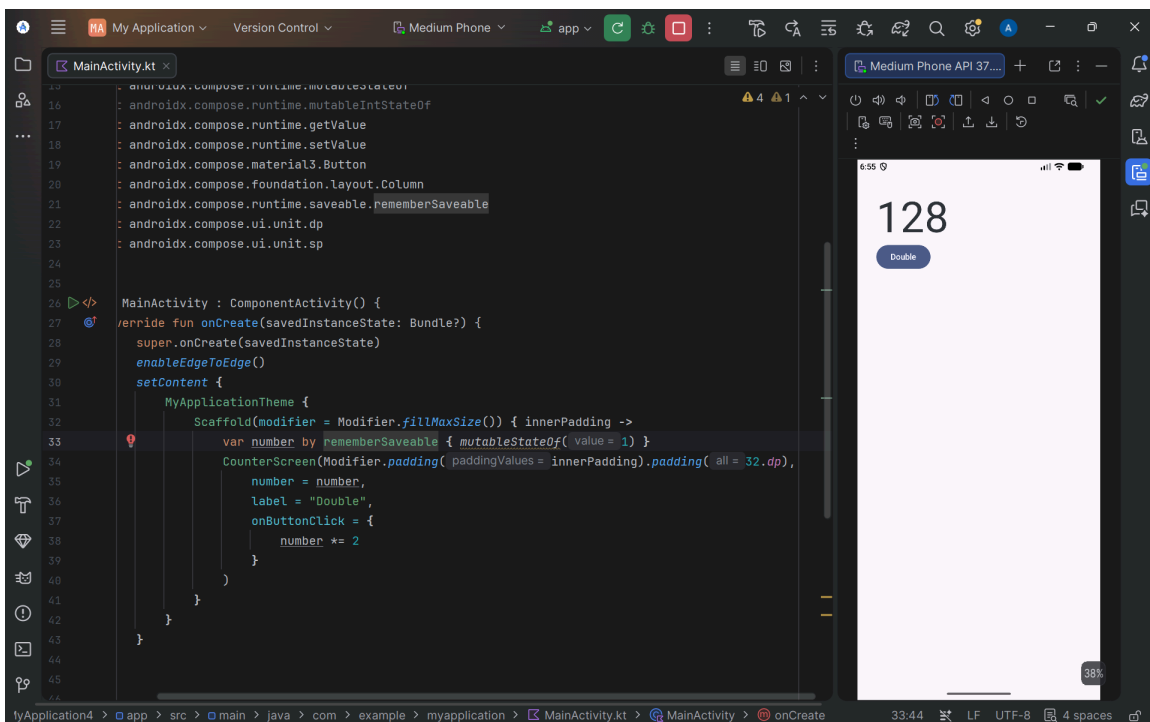
- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



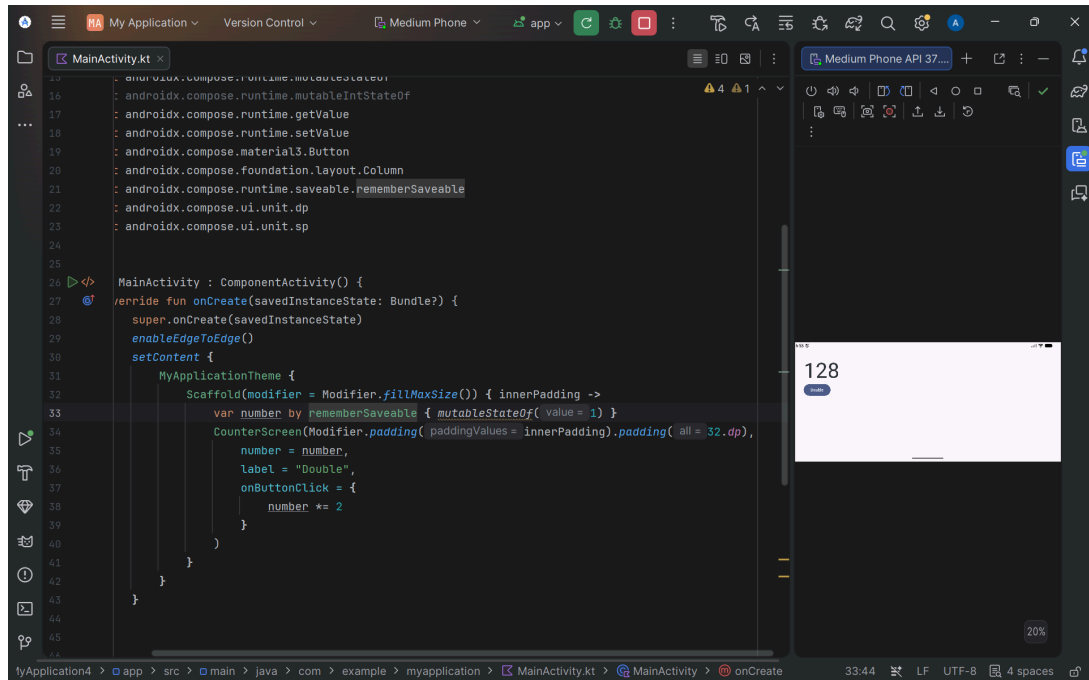
- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Karena remember hanya mempertahankan state selama proses recomposition. Saat konfigurasi berubah, misalnya orientasi layar berubah, Activity dapat dibuat ulang. Akibatnya: mutableStateOf(1) dijalankan kembali dan nilai state kembali menjadi 1.

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.



- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Karena rememberSaveable menyimpan nilai state menggunakan mekanisme saved state Android. Ketika Activity dibuat ulang, nilai sebelumnya dapat dipulihkan.

- c. Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember?

remember menyimpan state selama recomposition, tetapi nilainya dapat hilang ketika Activity dibuat ulang, misalnya saat rotasi layar. Sementara itu, rememberSaveable dapat mempertahankan state setelah perubahan konfigurasi, sehingga lebih cocok untuk data ringan seperti isi TextField, checkbox, nilai counter, atau posisi scroll.

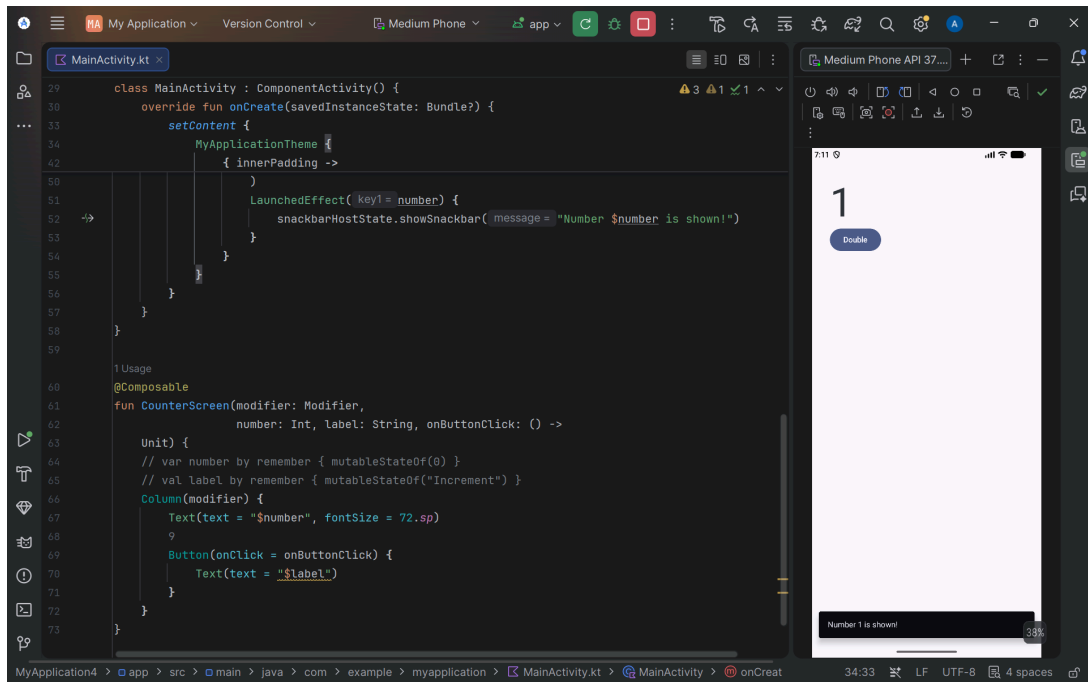
4. Beri contoh kasus nyata lain di mana penggunaan konstruk rememberSavable ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh penggunaan rememberSaveable adalah pada form pendaftaran ketika pengguna sudah mengisi nama atau email. Jika layar diputar dari portrait ke landscape, data yang sudah diketik tetap tersimpan dan tidak kembali kosong. Manfaatnya adalah menjaga data input pengguna saat terjadi perubahan konfigurasi sehingga pengguna tidak perlu mengisi ulang form dari awal.

Launched Effect

1. Jalankan program

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena:

`LaunchedEffect(number)`

akan dijalankan ketika composable pertama kali masuk ke composition. Jadi tidak perlu menunggu pengguna menekan tombol.

- c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Yang memicunya adalah:

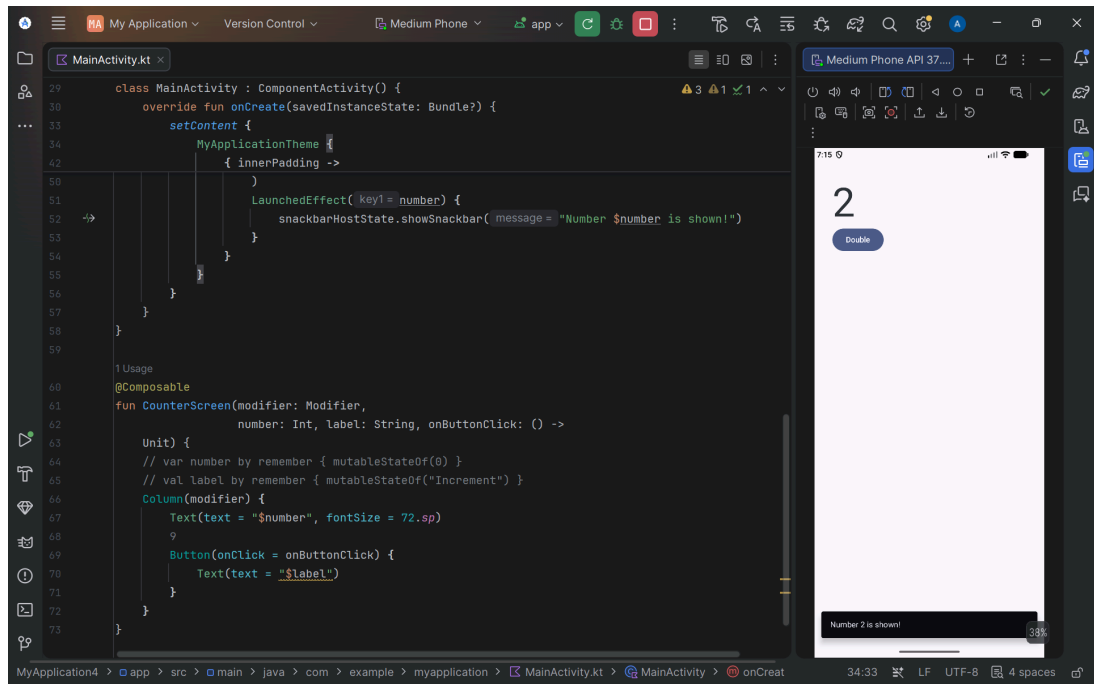
`LaunchedEffect(number)`

Nilai number digunakan sebagai key. Jika key berubah, efek akan dijalankan kembali.

2. Klik button "Double" satu kali,

- a. Apa yang terjadi?

Jika tombol ditekan satu kali, kemudian Snackbar berubah menjadi: Number 2 is shown!



b. Mengapa hal tersebut terjadi? Jelaskan!

Karena tombol menjalankan:

`number *= 2`

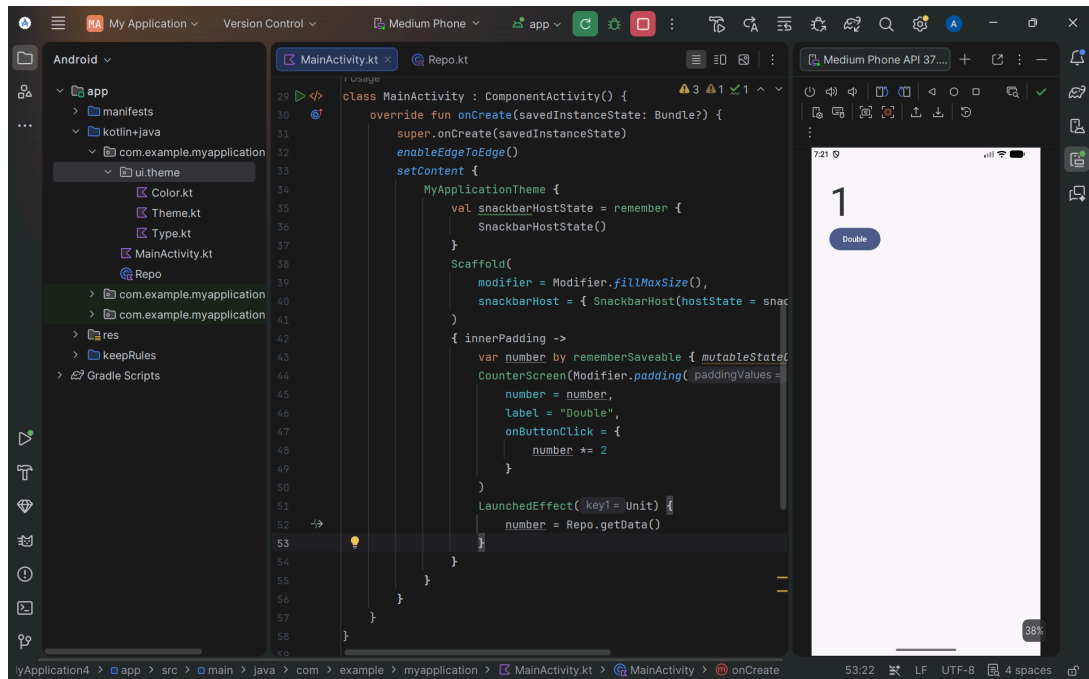
Akibatnya nilai number berubah. Karena number adalah key dari:

`LaunchedEffect(number)`

maka ketika number berubah, blok tersebut dijalankan kembali.

3. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?**

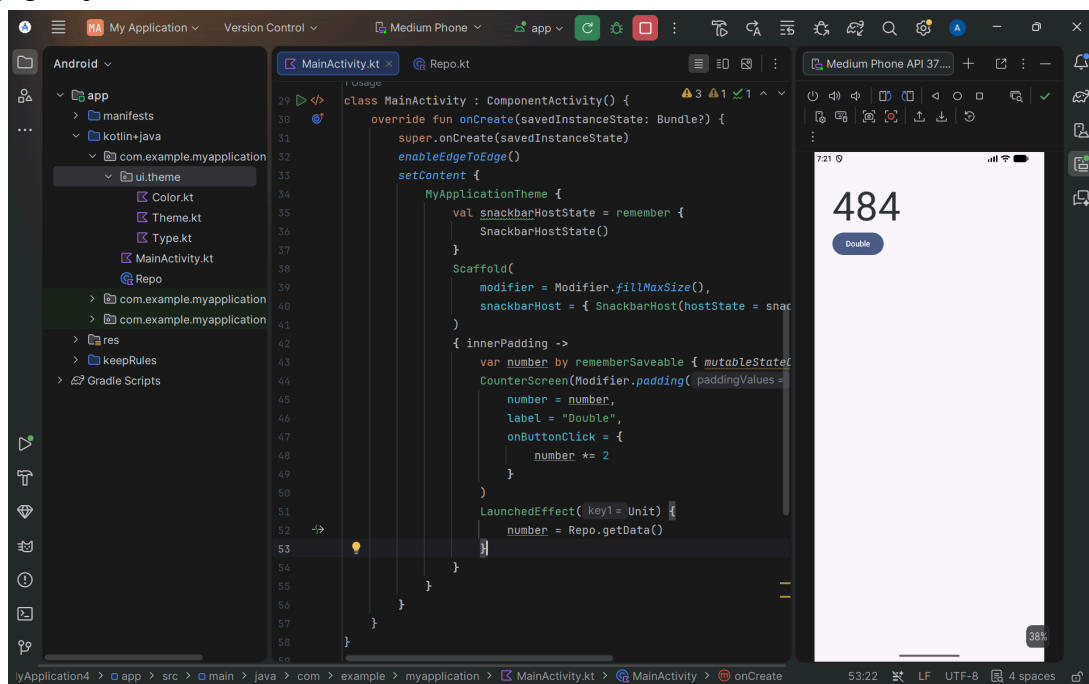


- b. **Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?**

Setelah dua detik, layar menampilkan angka baru. Karena number berubah, maka:

`LaunchedEffect(number)`

juga dijalankan kembali.



- c. **Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?**

Perubahan angka dipicu oleh:

```
LaunchedEffect(Unit) {
    number = Repo.getData()
```

```
}
```

Setelah dua detik:

```
Repo.getData()
↓
menghasilkan angka
↓
number berubah
```

Perubahan number kemudian memicu:

```
LaunchedEffect(number)
```

yang menampilkan Snackbar.

4. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect, a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?

1) LaunchedEffect(state)

Contoh:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar(
        "Number $number is shown!"
    )
}
```

Efek dijalankan:

- saat pertama kali masuk composition,
- ketika nilai number berubah.

Misalnya:

```
number = 1
↓
effect dijalankan

number = 2
↓
effect dijalankan

number = 4
↓
effect dijalankan
```

Jadi LaunchedEffect(number) cocok ketika sebuah proses bergantung pada perubahan state tertentu.

2) LaunchedEffect(Unit)

Contoh:

```
LaunchedEffect(Unit) {  
  
    number = Repo.getData()  
}
```

Unit adalah nilai yang tetap. Karena key-nya tidak berubah, blok tersebut tidak dijalankan ulang hanya karena state lain berubah. Biasanya digunakan untuk proses yang perlu dilakukan sekali ketika composable muncul, misalnya:

- load data
- memanggil API
- mengambil konfigurasi awal

b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?

Operasi di dalam LaunchedEffect(state) dijalankan saat composable pertama kali masuk ke composition dan akan dijalankan kembali setiap kali nilai state yang digunakan sebagai key berubah. Sementara itu, LaunchedEffect(Unit) dijalankan saat composable pertama kali masuk ke composition dan umumnya hanya sekali selama composable tersebut tetap berada di composition, karena nilai Unit tidak berubah.